

CHƯƠNG 7

GIÁM SÁT HỆ THỐNG MÁY CHỦ

Mục tiêu chương

Sau khi học xong chương này, người học có thể:

- Trình bày được ý nghĩa của giám sát hệ thống.
- Phân biệt được mô hình Pull và Push, và vai trò của Metrics trong giám sát.
- Thực hành thiết lập Prometheus thu thập metrics từ server và container.
- Thực hành thiết kế dashboard giám sát chuyên nghiệp trên Grafana.
- Phân tích và đưa ra quyết định dựa trên dữ liệu giám sát.

7.1 Tổng quan về giám sát máy chủ

7.1.1 Tại sao cần giám sát

Trong kỷ nguyên số hóa, hệ thống thông tin không chỉ là công cụ hỗ trợ mà đã trở thành huyết mạch sống còn của doanh nghiệp, từ các ứng dụng web, dịch vụ trực tuyến đến hạ tầng mạng và cơ sở dữ liệu. Một hệ thống ổn định không chỉ giúp tăng hiệu suất công việc mà còn là yếu tố quyết định trải nghiệm người dùng đối với sản phẩm. Do đó, giám sát (monitoring) không phải là một tùy chọn thêm thắt, mà là một phần quan trọng không thể thiếu của cơ sở hạ tầng hiện đại.

Phân tích các nguy cơ khi thiếu hệ thống giám sát: nếu không có một cơ chế theo dõi chặt chẽ, doanh nghiệp sẽ phải đối mặt với "sự cố ẩn" và hiệu suất thấp. Chúng ta thường không thể nhận biết được các vấn đề tiềm ẩn như tốc độ xử lý chậm lại theo thời gian, tài nguyên bị chiếm dụng quá mức một cách âm thầm, hay thời gian đáp ứng (latency) dài hơn mức cho phép cho đến khi hệ thống thực sự sụp đổ. Việc thiếu dữ liệu ghi lại hiệu suất khiến các nhà quản trị mù mờ về tình trạng thực tế, dẫn đến các sự cố không lường trước gây đình trệ toàn bộ quy trình kinh doanh.

Bên cạnh đó, khả năng dự báo và lập kế hoạch sẽ cực kỳ yếu kém. Dữ liệu giám sát cung cấp một bức tranh về xu hướng tăng trưởng và biến đổi tài nguyên. Thiếu thông tin này, việc lập kế hoạch mở rộng hạ tầng (capacity planning) chỉ dựa trên cảm tính, dẫn đến hai kịch bản tiêu cực: lãng phí tài nguyên khi đầu tư quá mức hoặc hệ thống quá tải vì không đáp ứng kịp nhu cầu thay đổi của thị trường. Ngoài ra, khi sự cố xảy ra, quá trình điều tra nguyên nhân gốc rễ (root cause analysis) sẽ gặp trở ngại lớn do thiếu dữ liệu lịch sử về hoạt động của hệ thống trước, trong và sau thời điểm phát sinh lỗi. Điều này làm kéo dài thời gian gián đoạn dịch vụ, trực tiếp làm giảm sự hài lòng của khách hàng và uy tín thương hiệu.

Tầm quan trọng trong an ninh và vận hành chủ động: giám sát đóng vai trò như một lớp bảo mật bổ sung quan trọng. Bằng cách theo dõi nhật ký máy chủ, lưu lượng mạng và hành vi hệ thống, nhà quản trị có thể phát hiện sớm các cố gắng truy cập trái phép hoặc các hoạt động bất thường mang tính rủi ro. Một hệ thống giám sát tốt cho

phép thiết lập các ngưỡng (thresholds) và cảnh báo tự động, giúp phát hiện sự cố ngay khi chúng bắt đầu nhen nhóm thay vì đợi đến khi người dùng khiếu nại.

Chúng ta xem xét case study về ”mô hình quản lý nhà hàng”: hãy coi việc vận hành một hệ thống máy chủ giống như điều hành một nhà hàng đông khách.

Kịch bản không giám sát: nhân viên bếp không kiểm tra lượng nguyên liệu trong kho. Họ cứ mãi miết nấu cho đến khi khách gọi món ”bò bít tết” và lúc đó mới phát hiện thịt bò đã hết sạch. Khách hàng không hài lòng, phải đợi lâu hoặc bỏ đi.

Kịch bản có giám sát: nhà hàng cài đặt hệ thống cảm biến hoặc bảng theo dõi kho thời gian thực. Khi lượng thịt bò xuống dưới mức 5kg, hệ thống tự động thông báo cho người quản lý. Quản lý chủ động đặt hàng mới trước khi nguyên liệu hết hẳn. Kết quả là nhà hàng hoạt động trơn tru, khách hàng luôn được phục vụ đúng lúc và hài lòng. Hệ thống giám sát máy chủ chính là ”bộ cảm biến kho” đó, giúp chúng ta biết trang web tải nhanh hay chậm, người dùng có gặp lỗi thanh toán hay không, và có bao nhiêu khách truy cập cùng lúc để điều phối tài nguyên hợp lý.

Như vậy ta có thể thấy những lợi ích của việc giám sát như:

- Giám sát giúp chuyển đổi từ trạng thái phản ứng thụ động (reactive) sang chủ động phòng ngừa (proactive).
- Dữ liệu giám sát là nền tảng cho việc tối ưu hóa chi phí và lập kế hoạch hạ tầng dài hạn.
- Hệ thống có khả năng quan sát được (observability) là yếu tố sống còn cho các mô hình cloud-native và microservices phức tạp.

7.1.2 Các tiêu chí cần theo dõi, thu thập trên máy chủ

Để xây dựng một hệ thống giám sát hiệu quả, việc xác định đúng các chỉ số (metrics) cần thu thập là bước tiên quyết. Một trong những phần mềm quản lý và theo dõi hiệu suất máy chủ được sử dụng rất phổ biến hiện nay là Prometheus, phần mềm này tập trung vào việc thu thập các metrics dạng chuỗi thời gian (time-series) – dữ liệu có cấu trúc giúp theo dõi trạng thái hệ thống theo từng mốc thời gian cụ thể.

Hiệu suất hạ tầng (Infrastructure Metrics): đây là nhóm chỉ số cơ bản nhất để đánh giá ”sức khỏe” của máy chủ vật lý hoặc máy chủ ảo (VM/container).

- **CPU Usage:** theo dõi mức độ sử dụng CPU tổng thể và chi tiết theo các nhãn (labels) như `user`, `system`, `iowait`. Chỉ số `iowait` cao thường cho thấy ổ đĩa đang gặp nghẽn cổ chai, trong khi CPU liên tục ở mức 90–100% là dấu hiệu hệ thống cần được nâng cấp hoặc tối ưu mã nguồn.
- **Memory (RAM) Usage:** theo dõi lượng RAM đang sử dụng so với tổng dung lượng. Cần chú ý đến bộ nhớ đệm (cache) và bộ nhớ swap; việc hệ thống bắt đầu dùng nhiều swap là dấu hiệu RAM đã cạn kiệt, dẫn đến tốc độ xử lý chậm trầm trọng.
- **Disk Capacity & I/O:** theo dõi dung lượng ổ đĩa còn trống để tránh tình trạng hệ thống bị ngưng trệ khi đầy disk. Đồng thời, các thông số đọc/ghi (Read/Write IOPS) giúp phát hiện các ứng dụng đang thao tác dữ liệu quá mức gây quá tải phần cứng.
- **Network Throughput:** đo lường băng thông tiêu thụ, tỷ lệ lỗi gói tin và độ trễ mạng để đảm bảo kết nối giữa máy chủ và người dùng luôn thông suốt.

Trạng thái ứng dụng (Application Metrics): thay vì chỉ xem máy chủ có chạy hay không, chúng ta cần đi sâu vào hiệu suất của chính ứng dụng đang vận hành.

- **Request Count:** tổng số lượng yêu cầu (HTTP requests) đổ về hệ thống mỗi giây. Chỉ số này giúp xác định tải trọng thực tế mà ứng dụng đang gánh vác.
- **Error Rate:** tỷ lệ các yêu cầu thất bại (ví dụ: HTTP 500 cho lỗi server, 404 cho lỗi không tìm thấy). Việc lỗi 5xx tăng đột biến là tín hiệu khẩn cấp về lỗi logic ứng dụng hoặc mất kết nối cơ sở dữ liệu.
- **Latency (Response Time):** thời gian trung bình để ứng dụng xử lý xong một yêu cầu. Đây là chỉ số quan trọng nhất ảnh hưởng trực tiếp đến trải nghiệm người dùng. Một ứng dụng có thể "sống" (up) nhưng nếu phản hồi mất 10 giây thì nó coi như đã "chết" đối với người dùng.

Mô hình dữ liệu đa chiều (Multi-dimensional Data Model): Prometheus sử dụng một định dạng dữ liệu đặc biệt giúp việc phân tích trở nên cực kỳ linh hoạt thông qua các nhãn (labels). Định dạng dữ liệu có cấu trúc:

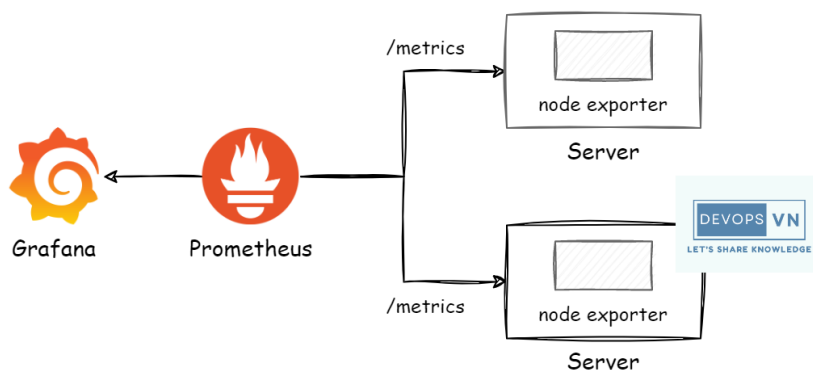
```
<metric name>{<label name>=<label value>, ...} <samples>
```

Ví dụ:

```
http_requests_total{method="POST", endpoint="/login", status="500"}
```

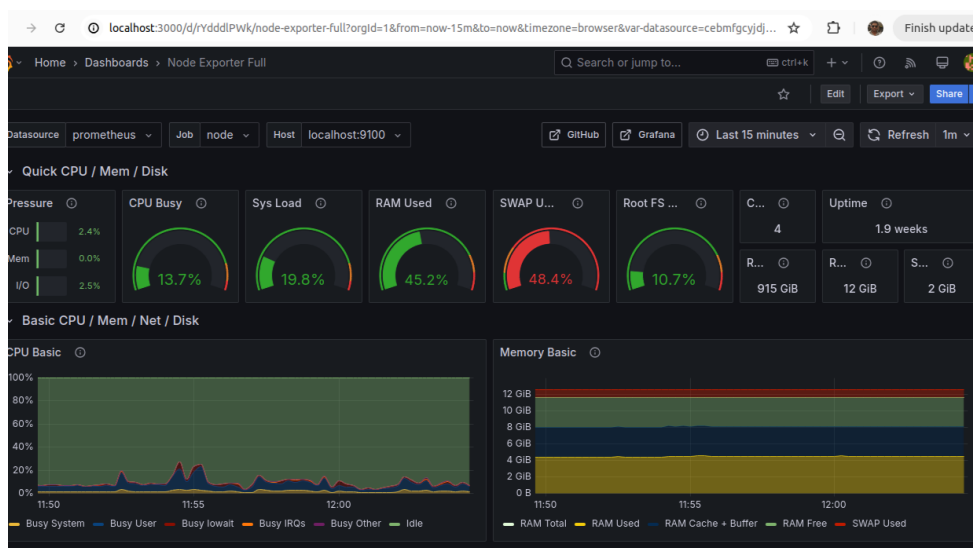
Phân tích: nhờ vào các nhãn `method`, `endpoint`, và `status`, nhà quản trị không chỉ biết tổng số yêu cầu mà còn có thể truy vấn chính xác: "Có bao nhiêu yêu cầu POST vào trang login bị lỗi 500?". Khả năng này cho phép lọc và tổng hợp dữ liệu theo nhiều góc nhìn khác nhau một cách nhanh chóng.

Hướng dẫn thực hành ngắn để bắt đầu thu thập các chỉ số này trên Linux, công cụ phổ biến nhất là Node Exporter. Khi được cài đặt, nó sẽ tự động expose một endpoint tại <http://localhost:9100/metrics>, cung cấp hàng trăm chỉ số về phần cứng cho Prometheus "scrape" về.



Hình 7.1: Cấu trúc của Node Exporter hoạt động

Sau khi thu thập thông tin trên máy chủ, Node Exporter sẽ hiển thị các chỉ số phần cứng về máy chủ như dưới đây:



Hình 7.2: Thông số dữ liệu hiển thị trên phần mềm Node Exporter

7.1.3 Mô hình Pull & Push

Một trong những đặc điểm kiến trúc quan trọng nhất khiến Prometheus khác biệt với nhiều hệ thống giám sát truyền thống chính là cơ chế thu thập dữ liệu. Prometheus chủ yếu hoạt động theo mô hình Pull (kéo), nhưng vẫn hỗ trợ mô hình Push (đẩy) cho các trường hợp đặc biệt.

Mô hình Pull (kiểu kéo): đây là cơ chế mặc định và là sức mạnh cốt lõi của Prometheus. Trong mô hình này, Prometheus Server đóng vai trò chủ động. Định kỳ (theo cấu hình `scrape_interval`), Prometheus sẽ gửi yêu cầu HTTP đến các mục tiêu (targets) để lấy (scrape) dữ liệu metrics về lưu trữ.

- **Ưu điểm về quyền kiểm soát:** server giám sát chủ động quyết định khi nào và với tần suất bao nhiêu thì lấy dữ liệu. Điều này ngăn chặn tình trạng server bị "ngập lụt" dữ liệu nếu các ứng dụng đồng loạt đẩy thông tin về quá nhiều cùng lúc.
- **Dễ dàng kiểm tra (health check):** nếu Prometheus không thể kéo dữ liệu từ một target, nó có thể xác định ngay lập tức target đó đang "down" thông qua chỉ số `up = 0`.
- **Đơn giản hóa cấu hình target:** các máy chủ hoặc ứng dụng chỉ cần mở một endpoint HTTP chứa metrics thô (dạng text), không cần biết Prometheus nằm ở đâu hay cần phải cấu hình IP của server giám sát phức tạp.
- **Service Discovery:** pull model hoạt động hoàn hảo trong môi trường động như Kubernetes. Prometheus có thể tự động tìm thấy các container mới và bắt đầu kéo dữ liệu mà không cần cấu hình thủ công cho từng thực thể.

Mô hình Push (kiểu đẩy): dù Pull là chủ đạo, nhưng có những kịch bản mà Prometheus không thể trực tiếp kéo dữ liệu. Ví dụ như các short-lived jobs (các tác vụ chạy trong thời gian ngắn như script sao lưu dữ liệu, dọn dẹp hệ thống) có thể hoàn thành và tắt trước khi Prometheus kịp thực hiện lượt scrape tiếp theo.

- **Cơ chế hoạt động:** các ứng dụng hoặc job sẽ chủ động gửi dữ liệu về một thành phần trung gian gọi là Pushgateway. Sau đó, Prometheus sẽ thực hiện việc "kéo" dữ liệu từ Pushgateway này về máy chủ chính của mình.

- **Ứng dụng thực tế:** ngoài các job ngắn hạn, mô hình Push còn hữu ích khi các máy chủ nằm sau tường lửa (firewall) ngăn chặn các kết nối đi vào từ bên ngoài, nhưng vẫn cho phép đẩy dữ liệu đi ra.

So sánh và phân tích chuyên sâu: mô hình Pull thường được coi là hiện đại và phù hợp hơn cho các hạ tầng microservices vì tính tự chủ của server giám sát. Tuy nhiên, một kiến trúc giám sát toàn diện thường là sự kết hợp linh hoạt: sử dụng Pull cho toàn bộ các service, máy chủ, container đang chạy liên tục, và dùng Push thông qua Pushgateway cho các tiến trình nghiệp vụ ngắt quãng.

Phân tích case study thực tế: trong một hệ thống Kubernetes, khi một pod (container) bị lỗi và tự khởi động lại liên tục, cơ chế Service Discovery của Prometheus sẽ tự động cập nhật địa chỉ IP mới của pod đó. Nhờ mô hình Pull, Prometheus vẫn tiếp tục kéo được dữ liệu mà không cần bất kỳ sự can thiệp thủ công nào từ quản trị viên. Nếu dùng mô hình Push truyền thống, mỗi khi pod thay đổi IP, chúng ta có thể gặp rắc rối lớn trong việc định tuyến dữ liệu về server trung tâm.

Một số lưu ý quan trọng:

- Pull là lựa chọn tối ưu cho sự ổn định, khả năng mở rộng và giám sát các dịch vụ chạy liên tục.
- Push là giải pháp bổ trợ cần thiết cho các tác vụ ngắn hạn hoặc các môi trường bị giới hạn về kết nối mạng.
- Hiểu rõ sự khác biệt này giúp quản trị viên thiết kế được hệ thống giám sát chịu tải tốt và không bỏ sót thông tin quan trọng.

7.2 Các giải pháp giám sát tập trung

7.2.1 Giới thiệu về Prometheus

Prometheus là một hệ thống giám sát và cảnh báo mã nguồn mở được thiết kế chuyên biệt cho việc thu thập và xử lý các chỉ số (metrics) từ ứng dụng và hạ tầng kỹ thuật số. Được khởi xướng bởi SoundCloud và sau đó trở thành dự án thứ hai tham gia Cloud Native Computing Foundation (CNCF) sau Kubernetes, Prometheus đã khẳng định vị thế là tiêu chuẩn vàng trong giám sát các hệ thống phân tán và hiện đại. Điểm khác biệt cốt lõi của Prometheus so với các công cụ truyền thống là nó tập trung hoàn toàn vào dữ liệu chuỗi thời gian (time-series metrics) thay vì nhật ký sự kiện (logs).

Chức năng chính mà Prometheus hỗ trợ gồm:

- Thu thập và lưu trữ dữ liệu dạng time-series metrics (CPU, RAM, request, latency...).
- Hỗ trợ truy vấn bằng PromQL để phân tích theo thời gian.
- Tích hợp Alertmanager để cấu hình cảnh báo khi vượt ngưỡng (ví dụ CPU cao, pod restart liên tục).
- Hỗ trợ nhiều exporter như node-exporter, kube-state-metrics để giám sát hạ tầng Kubernetes.

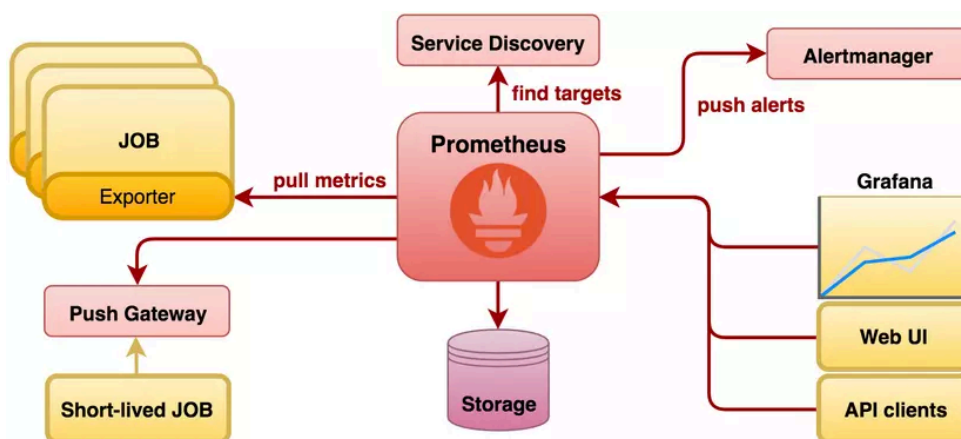
Về mặt triết lý thiết kế, Prometheus ưu tiên tính tin cậy và khả năng hoạt động độc lập. Mỗi máy chủ Prometheus là một thực thể tự chủ, lưu trữ dữ liệu cục bộ trong một cơ sở dữ liệu chuỗi thời gian (TSDB) tùy chỉnh mà không phụ thuộc vào các hệ thống lưu trữ phân tán phức tạp. Điều này đảm bảo rằng ngay cả khi hệ thống mạng hoặc các

thành phần hạ tầng khác gặp sự cố, Prometheus vẫn có thể hoạt động để cung cấp dữ liệu về tình trạng lỗi, giúp quản trị viên đưa ra quyết định chính xác.

Mô hình dữ liệu của Prometheus là một trong những điểm mạnh vượt trội, cho phép phân tích dữ liệu đa chiều một cách linh hoạt. Dữ liệu không chỉ đơn thuần là các con số mà được định danh bằng một "notation" bao gồm tên metric và các cặp nhãn (labels) key-value. Ví dụ, thay vì chỉ có một metric tổng về yêu cầu HTTP, Prometheus cho phép phân loại chi tiết như bên dưới. Khả năng này giúp người dùng dễ dàng thực hiện các phép tính tổng hợp hoặc lọc dữ liệu theo từng góc độ nghiệp vụ khác nhau.

```
http_requests_total{method="GET", endpoint="/api", status="200"}
```

Cơ chế thu thập dữ liệu của Prometheus dựa trên mô hình Pull (kéo). Hệ thống sẽ định kỳ gửi các yêu cầu HTTP (gọi là "scrape") tới các mục tiêu (targets) đã được định nghĩa để lấy dữ liệu về. Tần suất scrape thường dao động từ 10 đến 60 giây tùy vào cấu hình. Mô hình này giúp Prometheus kiểm soát tốt tải trọng hệ thống và dễ dàng quản lý việc phát hiện dịch vụ (Service Discovery) trong các môi trường động như Kubernetes hoặc Cloud, nơi các dịch vụ có thể thay đổi liên tục.



Hình 7.3: Sơ đồ kiến trúc của Prometheus

Kiến trúc của Prometheus bao gồm nhiều thành phần phối hợp nhịp nhàng. Prometheus Server đóng vai trò trung tâm xử lý thu thập và lưu trữ. Các Exporters là những công cụ nhỏ được cài đặt cạnh ứng dụng để chuyển đổi dữ liệu sang định dạng Prometheus hiểu được. Pushgateway được sử dụng để hỗ trợ các tác vụ ngắn hạn (short-lived jobs) chủ động đẩy dữ liệu về. Cuối cùng là Alertmanager, thành phần chịu trách nhiệm nhận cảnh báo từ Prometheus server, sau đó nhóm, lọc và điều hướng thông báo tới các kênh như Email, Slack hay Telegram.

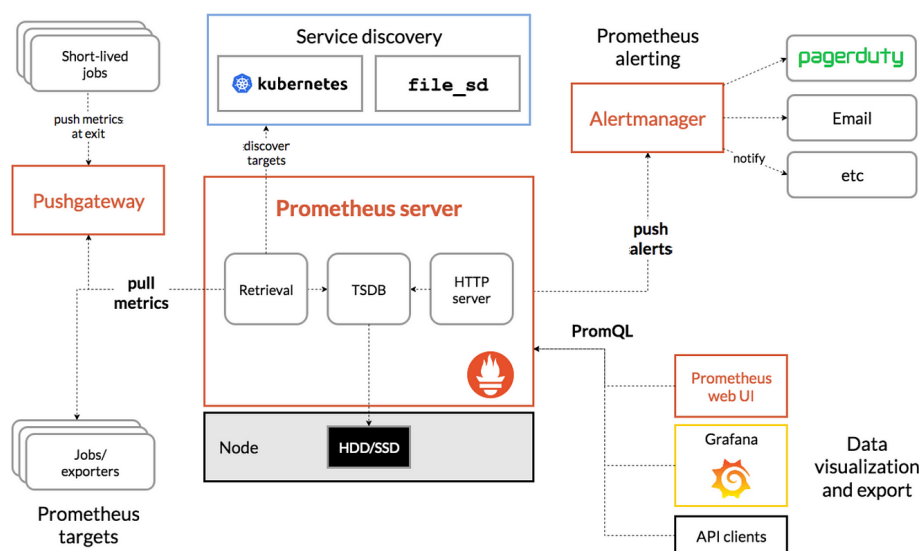
Để truy vấn dữ liệu, Prometheus cung cấp ngôn ngữ PromQL (Prometheus Query Language) cực kỳ mạnh mẽ. PromQL cho phép người dùng thực hiện các phép tính toán học, thống kê trên các chuỗi thời gian, chẳng hạn như tính tỷ lệ yêu cầu mỗi giây (rate), tính toán phần trăm sử dụng tài nguyên hoặc dự báo xu hướng đầy ổ đĩa. Đây là công cụ không thể thiếu để tạo ra các báo cáo hiệu suất chuyên sâu và thiết lập các quy tắc cảnh báo chính xác.

Khả năng lưu trữ và nén dữ liệu của Prometheus cũng rất ấn tượng. Phiên bản 2.0 có thể thu thập hàng triệu mẫu dữ liệu mỗi giây với mức nén đạt 1.3 byte cho mỗi mẫu

dữ liệu thực tế. Mặc dù thời gian lưu trữ mặc định là 15 ngày, nhưng người dùng có thể mở rộng thông qua các API remote read/write để kết nối với các hệ thống lưu trữ dài hạn như Thanos hay InfluxDB.

Về phương diện bảo mật và vận hành, Prometheus hỗ trợ tính năng Service Discovery để tự động tìm kiếm các mục tiêu giám sát trong cụm Kubernetes hoặc các nền tảng đám mây như AWS EC2. Điều này loại bỏ hoàn toàn việc cấu hình thủ công danh sách IP các máy chủ – một công việc vốn rất tốn thời gian và dễ sai sót trong các hạ tầng lớn.

Ngoài việc giám sát hạ tầng, Prometheus còn hỗ trợ Client Libraries cho nhiều ngôn ngữ lập trình như Java, Python, Go, Ruby. Các thư viện này cho phép lập trình viên tích hợp trực tiếp việc đo lường vào mã nguồn ứng dụng (direct instrumentation), giúp theo dõi các chỉ số nghiệp vụ như số lượng giao dịch thành công, số người dùng đang online hoặc thời gian xử lý của từng hàm cụ thể.



Hình 7.4: Sơ đồ tích hợp hệ thống giám sát Prometheus

Cuối cùng, Prometheus không chỉ là một công cụ giám sát mà còn là một giải pháp cảnh báo toàn diện. Thông qua các Alerting rules, hệ thống có thể tự động nhận diện các điều kiện bất thường (ví dụ: CPU > 90% trong 5 phút) và kích hoạt quy trình xử lý sự cố ngay lập tức. Điều này giúp doanh nghiệp giảm thiểu thời gian gián đoạn dịch vụ và nâng cao tính sẵn sàng của hệ thống.

Case study: chuyển đổi hệ thống giám sát tại một ngân hàng số. Một ngân hàng số tại Việt Nam từng sử dụng các công cụ giám sát kiểu cũ dựa trên SNMP và logs. Khi quy mô hệ thống tăng lên hàng ngàn microservices, các công cụ cũ gặp hiện tượng ”nghẽn cổ chai” và không thể theo dõi trạng thái các container thay đổi liên tục. Sau khi triển khai Prometheus kết hợp với Kubernetes Service Discovery, ngân hàng đã có thể:

- Tự động phát hiện và giám sát 100% các pod mới sinh ra.
- Giảm 70% các cảnh báo nhiễu nhờ cơ chế nhóm cảnh báo của Alertmanager.
- Phát hiện sớm các lỗi rò rỉ bộ nhớ (memory leak) của ứng dụng Java thông qua biểu đồ JVM metrics thu thập từ Micrometer.

Lưu ý: khi cấu hình Prometheus, hãy chú ý đến thông số `retention.time`. Nếu

ta lưu quá nhiều dữ liệu trong thời gian dài trên ổ đĩa tốc độ thấp, hiệu suất truy vấn PromQL sẽ giảm đáng kể.

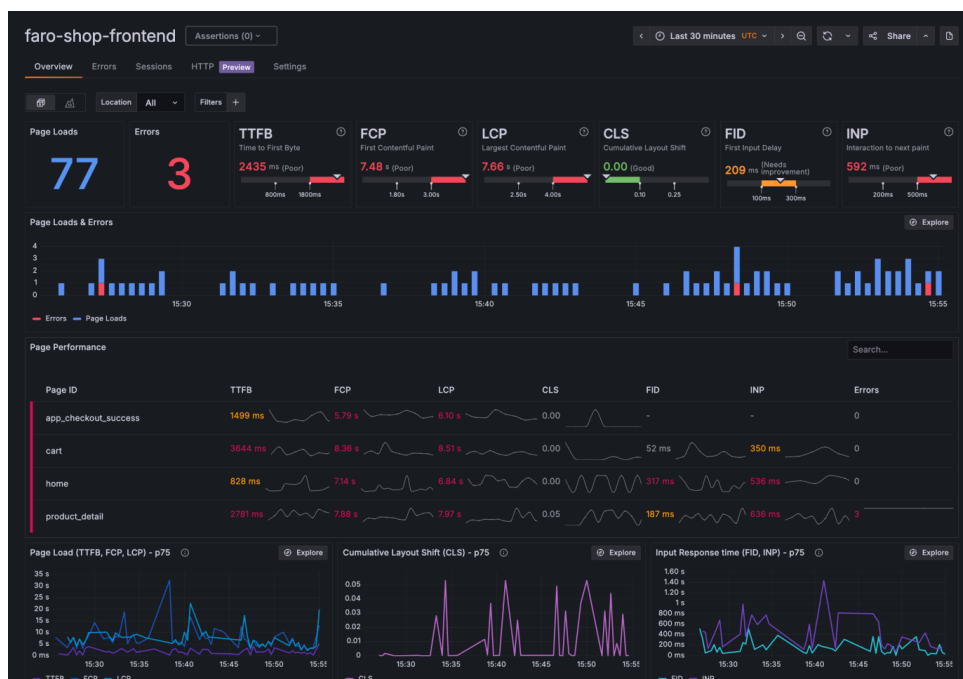
Sử dụng công cụ `promtool` để kiểm tra cú pháp file cấu hình `prometheus.yml` trước khi khởi động lại dịch vụ để tránh làm sập hệ thống giám sát do lỗi viết sai dấu cách (YAML syntax).

7.2.2 Giới thiệu Grafana

Grafana được mệnh danh là "đôi mắt" của hệ thống giám sát, là nền tảng mã nguồn mở hàng đầu thế giới về trực quan hóa và phân tích dữ liệu. Nếu Prometheus tập trung vào việc thu thập dữ liệu thô thì Grafana đóng vai trò biến những con số khô khan đó thành các bảng điều khiển (dashboards) tương tác, sinh động và dễ hiểu cho cả kỹ thuật viên lẫn các cấp quản lý.

Khả năng mạnh mẽ nhất của Grafana chính là sự linh hoạt trong việc kết nối với nhiều nguồn dữ liệu (data sources) khác nhau. Không chỉ giới hạn ở Prometheus, Grafana có thể đồng thời hiển thị dữ liệu từ MySQL, PostgreSQL, Elasticsearch, InfluxDB, AWS CloudWatch và thậm chí là các hệ thống nhật ký như Loki. Điều này cho phép quản trị viên tạo ra một "bức tranh toàn cảnh" (single pane of glass), nơi họ có thể nhìn thấy cả chỉ số hiệu suất phần cứng lẫn dữ liệu kinh doanh trên cùng một màn hình.

Giao diện của Grafana dựa trên các Panels, mỗi panel là một đơn vị hiển thị dữ liệu riêng biệt. Người dùng có thể tùy chọn rất nhiều kiểu hiển thị từ biểu đồ đường (graph), biểu đồ cột, đồng hồ đo (gauge), bảng số liệu cho đến biểu đồ nhiệt (heatmap). Khả năng tùy biến này giúp làm nổi bật những thông tin quan trọng nhất, ví dụ như sử dụng màu đỏ để cảnh báo khi một chỉ số vượt ngưỡng an toàn.



Hình 7.5: Giao diện của Grafana hiển thị các chỉ số giám sát

Một tính năng cực kỳ hữu ích khác là Variables (biến số). Thay vì tạo hàng chục dashboard cho hàng chục máy chủ, bạn chỉ cần tạo một dashboard duy nhất và sử dụng biến số để lọc dữ liệu theo từng máy chủ (instance), từng vùng (region) hoặc từng dịch

vụ. Khi chọn một giá trị từ danh sách thả xuống, toàn bộ các biểu đồ trên dashboard sẽ tự động cập nhật dữ liệu tương ứng, giúp tiết kiệm thời gian quản trị đáng kể.

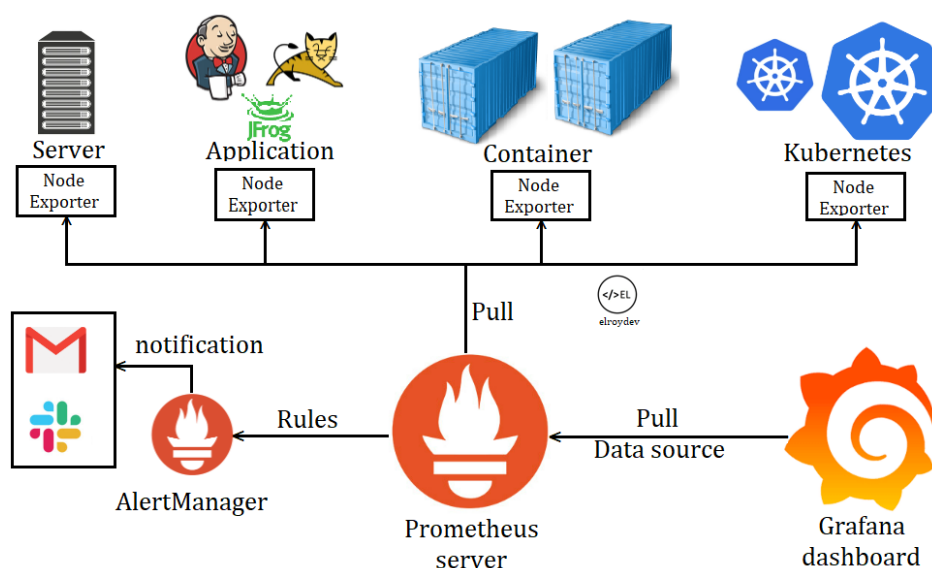
Grafana không chỉ để xem mà còn hỗ trợ Alerting Engine riêng. Mặc dù Prometheus có Alertmanager, nhưng Grafana cung cấp khả năng thiết lập cảnh báo dựa trên giao diện trực quan. Người dùng có thể vẽ một đường ngưỡng ngay trên biểu đồ và yêu cầu Grafana gửi thông báo qua Slack hay Discord khi dữ liệu chạm vào đường đó. Điều này rất phù hợp cho các cảnh báo mang tính trực quan nhanh mà không cần viết mã PromQL phức tạp.

Về mặt quản lý và cộng tác, Grafana hỗ trợ mô hình Role-Based Access Control (RBAC). Bạn có thể phân quyền cho các thành viên trong nhóm: ai có quyền chỉnh sửa dashboard (Admin/Editor), ai chỉ có quyền xem (Viewer). Tính năng này đảm bảo an toàn cho các cấu hình hệ thống quan trọng trong khi vẫn cho phép các bên liên quan theo dõi được trạng thái dịch vụ.

Hệ sinh thái Plugins của Grafana vô cùng phong phú với hàng trăm dashboard mẫu do cộng đồng đóng góp. Thay vì phải tự thiết kế từ đầu, quản trị viên có thể truy cập kho lưu trữ của Grafana, tìm kiếm các dashboard cho Linux (như ID 1860) hay Docker, sau đó chỉ cần nhập ID vào hệ thống để có ngay một giao diện chuyên nghiệp trong vài giây.

Bên cạnh các biểu đồ động, Grafana còn hỗ trợ Annotations (ghi chú). Khi hệ thống có một sự kiện quan trọng như triển khai phiên bản phần mềm mới (deploy) hoặc bảo trì định kỳ, bạn có thể tạo một ghi chú ngay trên dòng thời gian của biểu đồ. Điều này giúp các kỹ thuật viên dễ dàng liên hệ giữa việc thay đổi hệ thống và sự biến động của hiệu suất (ví dụ: ngay sau khi deploy thì CPU tăng vọt).

Trong các môi trường doanh nghiệp lớn, Grafana còn đóng vai trò là công cụ báo cáo hiệu suất (performance reporting). Dữ liệu từ Grafana có thể được xuất ra dưới dạng file PDF hoặc hình ảnh để gửi cho khách hàng hoặc cấp trên, chứng minh tính ổn định và sự sẵn sàng của dịch vụ (SLA).



Hình 7.6: Ví dụ về tích hợp Grafana với các hệ thống giám sát

Sự kết hợp hoàn hảo giữa Prometheus và Grafana tạo nên một hệ thống giám sát toàn diện. Prometheus chịu trách nhiệm ”hậu phương” với việc thu thập và lưu trữ dữ

liệu ổn định, còn Grafana là "tiền tuyến" thể hiện thông tin một cách thông minh, giúp con người có thể nhanh chóng nắm bắt tình trạng của hàng vạn máy chủ chỉ trong một cái nhìn lướt qua.

Case study: giám sát sàn thương mại điện tử trong ngày Black Friday. Một sàn thương mại điện tử lớn sử dụng Grafana để giám sát lưu lượng truy cập vào ngày hội mua sắm. Họ thiết lập một "war room" với các màn hình lớn hiển thị:

- Số lượng đơn hàng thành công theo thời gian thực (real-time orders).
- Tỷ lệ lỗi thanh toán từ các ngân hàng (payment gateway errors).
- Tải trọng của hệ thống CDN và các máy chủ web. Nhờ các biểu đồ này, khi một cổng thanh toán bị treo, đội ngũ kỹ thuật đã phát hiện ngay lập tức nhờ panel chuyển sang màu đỏ và kịp thời chuyển hướng luồng thanh toán sang cổng dự phòng, cứu vãn hàng tỷ đồng doanh thu.

Lưu ý khi triển khai:

- **Sai lầm thường gặp:** tạo quá nhiều biểu đồ trên cùng một dashboard dẫn đến tình trạng "ô nhiễm thông tin" (information overload), làm chậm tốc độ tải trang và gây rối mắt. Hãy chia nhỏ thành các dashboard chuyên biệt cho từng tầng (hạ tầng, database, application).
- **Lưu ý bảo mật:** luôn thay đổi mật khẩu mặc định của tài khoản admin/admin ngay sau khi cài đặt Grafana để tránh bị xâm nhập trái phép vào hệ thống giám sát.

Như vậy:

- Prometheus là "bộ não" thu thập dữ liệu chuỗi thời gian thông qua mô hình Pull và ngôn ngữ PromQL mạnh mẽ.
- Grafana là "giao diện" hiển thị dữ liệu trực quan, linh hoạt và hỗ trợ cộng đồng dashboard phong phú.
- Bộ đôi này giúp doanh nghiệp chuyển đổi từ giám sát thụ động sang quản trị hệ thống chủ động, phát hiện rủi ro trước khi nó trở thành sự cố.

7.3 Thực hành thiết lập cấu hình trên Prometheus

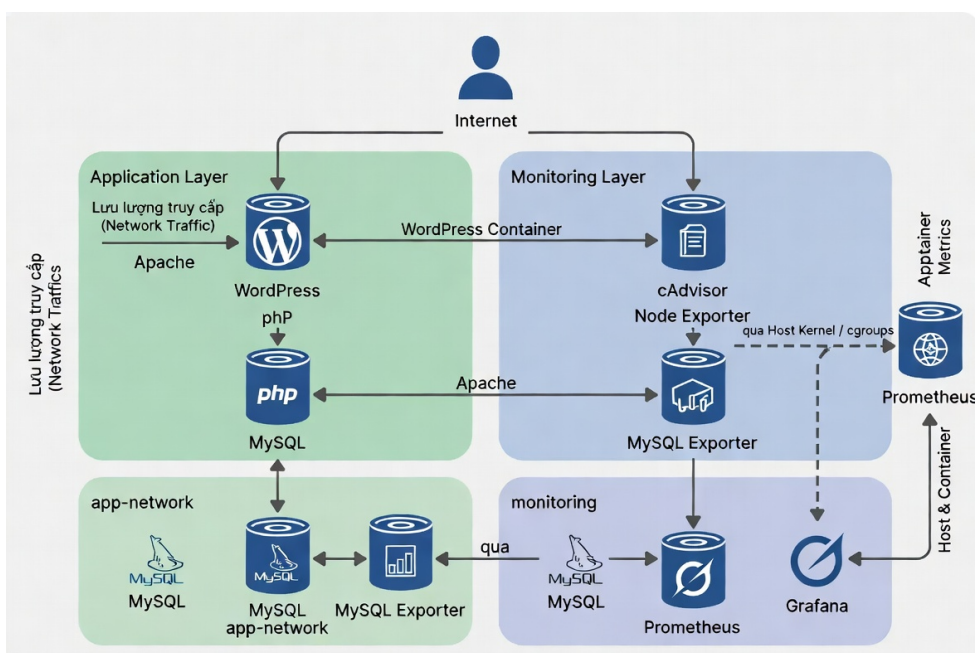
7.3.1 Kịch bản và cấu trúc hệ thống

Nội dung cài đặt và cấu hình hệ thống giám sát này sẽ thực hiện kịch bản sử dụng Prometheus để giám sát cho hệ thống website thương mại điện tử sử dụng CMS WordPress, kết hợp với CSDL MySQL.

Kiến trúc của hệ thống bao gồm các thành phần như sau:

- WordPress (web app): chạy trên Apache/Nginx + PHP.
- MySQL: DB riêng.
- Monitoring stack:
 - Prometheus: thu thập metrics.
 - Grafana: visualize (dashboard sẵn cho MySQL, Node, cAdvisor...).
 - cAdvisor: monitor resource container (CPU, RAM, network traffic...).
 - Node Exporter: monitor host metrics.
 - MySQL Exporter: metrics cho MySQL (queries, connections, replication...).
 - Nginx Exporter (nếu dùng Nginx) hoặc Apache Exporter (nếu dùng Apache) để monitor traffic (requests/sec, latency, status codes...).

Hệ thống sẽ được triển khai hoàn toàn trên docker compose, để đảm bảo an toàn cho hệ thống và dữ liệu ứng dụng, chúng ta sẽ sử dụng 2 network độc lập: 1 cho ứng dụng và 1 cho hệ thống monitoring. Sơ đồ triển khai như hình dưới:



Hình 7.7: Sơ đồ kết nối các thành phần của hệ thống

Trong đó:

- Màu xanh lá: layer ứng dụng (WordPress + MySQL).
- Màu xanh dương: layer giám sát (Prometheus stack).
- Đường nét liền: kết nối mạng bình thường.
- Đường nét đứt: thu thập metrics qua kernel/host (cAdvisor).
- MySQL Exporter: container đặc biệt nằm ở cả hai network.

7.3.2 Triển khai hệ thống với docker compose

Chúng ta sẽ cần tạo thư mục để lưu các tài nguyên của toàn bộ dự án, ví dụ `wordpress-monitoring`, tiếp theo cần tạo file `docker-compose.yml` chứa mã triển khai hệ thống:

```
services:
  # === Ứng dụng ===
  wordpress:
    image: wordpress:php8.2-apache
    container_name: wordpress
    restart: unless-stopped
    ports:
      - "8080:80"
    environment:
      WORDPRESS_DB_HOST: mysql:3306
      WORDPRESS_DB_USER: wpuser
      WORDPRESS_DB_PASSWORD: wppass
      WORDPRESS_DB_NAME: wpdb
    volumes:
```

```
- wordpress_data:/var/www/html
networks:
  - app-network
depends_on:
  - mysql

mysql:
  image: mysql:8.0
  container_name: mysql
  restart: unless-stopped
  environment:
    MYSQL_ROOT_PASSWORD: rootpass
    MYSQL_DATABASE: wpdb
    MYSQL_USER: wpuser
    MYSQL_PASSWORD: wppass
  volumes:
    - mysql_data:/var/lib/mysql
  networks:
    - app-network

# === Monitoring Stack ===
prometheus:
  image: prom/prometheus:latest
  container_name: prometheus
  restart: unless-stopped
  ports:
    - "9090:9090"
  volumes:
    - ./prometheus.yml:/etc/prometheus/prometheus.yml
    - prometheus_data:/prometheus
  command:
    - '--config.file=/etc/prometheus/prometheus.yml'
    - '--storage.tsdb.path=/prometheus'
    - '--web.console.libraries=/etc/prometheus/console_libraries'
    - '--web.console.templates=/etc/prometheus/consoles'
  networks:
    - monitoring

grafana:
  image: grafana/grafana:latest
  container_name: grafana
  restart: unless-stopped
  ports:
    - "3000:3000"
  volumes:
    - grafana_data:/var/lib/grafana
  networks:
    - monitoring
  depends_on:
    - prometheus

cadvisor:
  image: gcr.io/cadvisor/cadvisor:latest
  container_name: cadvisor
  restart: unless-stopped
  ports:
    - "8081:8080"
  volumes:
    - /:/rootfs:ro
```

```
- /var/run:/var/run:ro
- /sys:/sys:ro
- /var/lib/docker:/var/lib/docker:ro
networks:
  - monitoring

node-exporter:
  image: prom/node-exporter:latest
  container_name: node-exporter
  restart: unless-stopped
  ports:
    - "9100:9100"
  networks:
    - monitoring

mysql-exporter:
  image: prom/mysqld-exporter:latest
  container_name: mysql-exporter
  restart: unless-stopped
  ports:
    - "9104:9104"
  environment:
    DATA_SOURCE_NAME: "wpuser:wppass@tcp(mysql:3306)/wpdb"
  networks:
    - app-network
    - monitoring
  depends_on:
    - mysql

# Nếu dùng Nginx thay vì Apache mặc định của WordPress:
# nginx-exporter:
#   image: nginx/nginx-prometheus-exporter:latest
#   container_name: nginx-exporter
#   ports:
#     - "9113:9113"
#   command: ["-nginx.scrape-uri", "http://wordpress:80/stub_status"]
#   networks:
#     - app-network
#     - monitoring

volumes:
  wordpress_data:
  mysql_data:
  prometheus_data:
  grafana_data:

networks:
  app-network:
  monitoring:
```

Có thể thấy hệ thống sử dụng hai phân vùng mạng độc lập là: `app-network` và `monitoring`. Các service bao gồm:

- `wordpress`, `mysql`: hoạt động trên mạng `app-network`.
- `prometheus`, `grafana`, `cadvisor`, `node-exporter`, `mysql-exporter`: hoạt động trên mạng `monitoring`.

Điểm đặc biệt là service `mysql-exporter` hoạt động trên cả hai mạng đồng thời.

Điều này phản ánh vai trò làm cầu nối để thu thập dữ liệu của service này.

Cấu hình của Prometheus được thiết lập trong file `prometheus.yml` như sau:

```
global:
  scrape_interval: 15s

scrape_configs:
  - job_name: 'prometheus'
    static_configs:
      - targets: ['localhost:9090']

  - job_name: 'node-exporter'
    static_configs:
      - targets: ['node-exporter:9100']

  - job_name: 'cadvisor'
    static_configs:
      - targets: ['cadvisor:8080']

  - job_name: 'mysql'
    static_configs:
      - targets: ['mysql-exporter:9104']

  - job_name: 'wordpress' # Hoặc blackbox cho HTTP checks
    metrics_path: /metrics
    static_configs:
      - targets: ['wordpress:80'] # Nếu WordPress expose metrics (thường cần plugin
        ↪ hoặc exporter)

# - job_name: 'nginx'
#   static_configs:
#     - targets: ['nginx-exporter:9113']
```

Các từ khóa cấu hình trong file này được quy định và hiểu bởi hệ thống Prometheus khi vận hành, nếu muốn biết chi tiết và đầy đủ hơn, ta có thể tham khảo thêm tại trang chủ và tài liệu cấu hình của hệ thống này.

Sau khi tạo xong, chúng ta có thể khởi động các container bằng câu lệnh:

```
# Tạo mạng nếu cần
$docker compose up -d
```

Các đường link truy cập hệ thống gồm:

- Prometheus: **http://localhost:9090**
- Grafana: **http://localhost:3000** (user/pass mặc định: admin/admin)
- WordPress: **http://localhost:8080**

Trong Grafana, ta cần thiết lập cấu hình cho các service để có thể bắt đầu thu thập traffic mạng, các thông số tham khảo – import dashboard ID:

- MySQL: 7362 hoặc 11375
- Node Exporter: 1860
- cAdvisor / Docker: 14282 hoặc 193
- Nginx: 12740

Hãy kiểm tra và đảm bảo các container đã được khởi tạo và hoạt động, sử dụng câu lệnh:

```
$docker compose ps
```

7.3.3 Cơ chế hoạt động của các network trong hệ thống giám sát

Trong file `docker-compose.yml` đã đưa ra ở trên, chúng ta sử dụng 2 network riêng biệt: `app-network` và `monitoring`.

Lý do cần phải dùng 2 network đó là: đây là best practice về security và isolation, hãy xem bảng phân tích vai trò các network sau:

Bảng 7.1: Vai trò của hai network trong hệ thống giám sát

Mục đích	app-network	monitoring
Chứa service nào	WordPress + MySQL	Prometheus, Grafana, cAdvisor, Node-exporter, MySQL-exporter
Mục tiêu	Traffic ứng dụng (web ↔ DB)	Traffic monitoring
Lý do tách biệt	Giới hạn bề mặt tấn công	Monitoring tools không cần truy cập DB trực tiếp
Bảo mật	Cao (chỉ cho phép cần thiết)	Monitoring tools có thể expose metrics

Lợi ích chính:

- MySQL không bị expose ra ngoài mạng monitoring.
- Prometheus/Grafana không thể trực tiếp kết nối vào MySQL (tránh rủi ro bảo mật).
- Dễ quản lý và scale sau này.

Trong khi đó, `mysql-exporter` nằm trên cả hai network, quy trình hoạt động của service này như sau:

- `mysql-exporter` nằm trên cả 2 network.
- Nó dùng `DATA_SOURCE_NAME` để kết nối đến `mysql:3306` qua `app-network`.
- Prometheus (trên mạng `monitoring`) scrape metrics từ `mysql-exporter:9104`.

Đây gọi là cơ chế multi-network attachment – một container có thể join nhiều Docker network cùng lúc.

Cuối cùng, Prometheus thu thập metrics của WordPress & MySQL như sau:

Bảng 7.2: Cách Prometheus thu thập metrics từng service

Service	Cách Prometheus thu thập metrics	Network sử dụng
MySQL	Qua <code>mysql-exporter</code>	<code>monitoring</code> + <code>app-network</code>
WordPress	Gián tiếp qua cAdvisor (network I/O, CPU, RAM...)	<code>monitoring</code>
Host / Containers	Qua cAdvisor và Node Exporter	<code>monitoring</code>
Prometheus itself	Tự scrape	<code>monitoring</code>

Trong hệ thống này, cAdvisor đóng vai trò quan trọng trong việc thu thập các lưu lượng mạng của các container từ host. Mặc dù cAdvisor chỉ join network monitoring, không join app-network. Nhưng cAdvisor vẫn thu thập được network traffic của WordPress và MySQL là nhờ cơ chế hoạt động đặc biệt của nó.

cAdvisor không thu thập metrics qua Docker Network. Lí do là Docker Network (bridge, overlay...) chỉ dùng để các container giao tiếp với nhau (ví dụ WordPress gọi MySQL qua `mysql:3306`). Nhưng metrics về network traffic (bytes nhận/gửi, packets...) không phải được lấy qua việc "kết nối mạng" giữa các container.

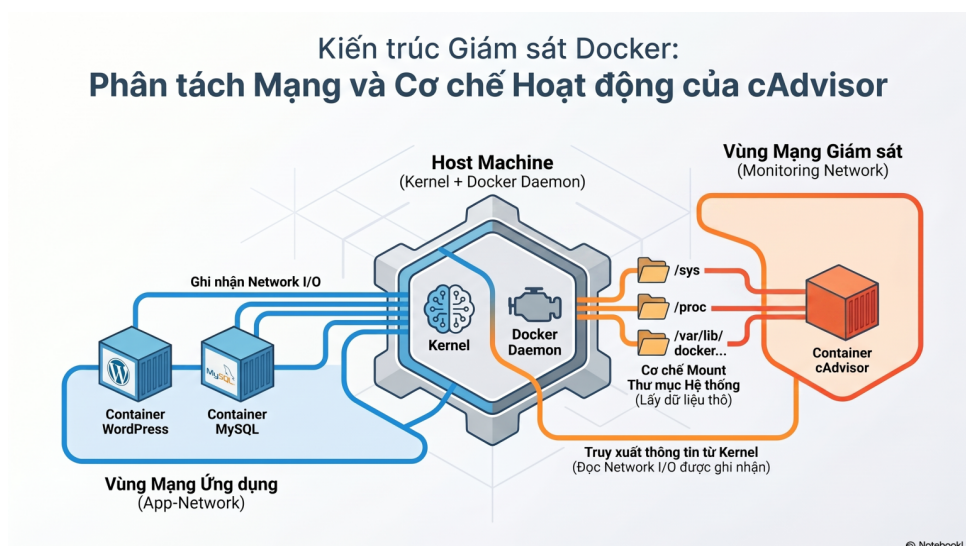
cAdvisor thu thập metrics qua cgroup và Docker Daemon (ở mức host). Đây chính là lý do quan trọng: khi chạy cAdvisor, chúng ta mount nhiều thư mục từ host vào container, trong file compose có thể thấy nội dung cấu hình sau:

```
volumes:
- /:/rootfs:ro
- /var/run:/var/run:ro
- /sys:/sys:ro
- /var/lib/docker:/var/lib/docker:ro
```

Tức là cAdvisor chạy ở mức kernel/host, nó đọc trực tiếp từ:

- `/sys/fs/cgroup` → kernel theo dõi tài nguyên của mọi container (CPU, Memory, Network I/O, Disk I/O).
- Docker socket (`/var/run/docker.sock`) → để biết thông tin tất cả container đang chạy (tên, ID, labels...).

Do đó, cAdvisor thấy được tất cả container trên cùng một Docker Host, bất kể chúng đang ở network nào. Ta có thể thấy chi tiết qua sơ đồ dưới đây:

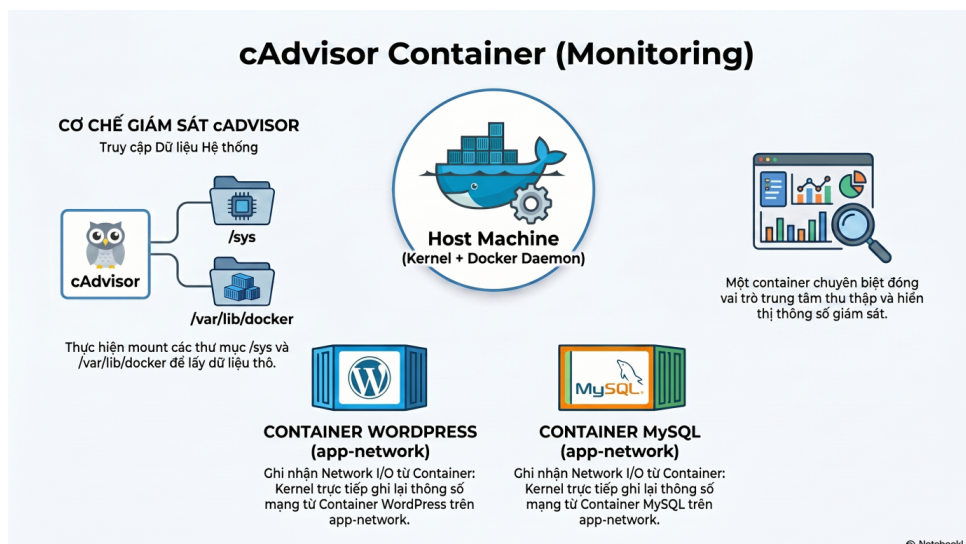


Hình 7.8: Cơ chế thu thập traffic của cAdvisor

Hay cách chi tiết hơn:

7.3.4 Tích hợp với Grafana

Để tích hợp các dịch vụ vào Grafana, ta có thể thực hiện việc thêm các source vào hệ thống để bắt đầu thu thập và hiển thị dữ liệu trên dashboard.



Hình 7.9: Sơ đồ chi tiết cách hoạt động của cAdvisor

Để thêm Prometheus làm Data Source, ở menu bên trái, chọn Configuration (biểu tượng bánh răng) → Data Sources.

- Nhấn Add data source.
- Chọn Prometheus.

Điền thông tin cấu hình:

- Name: Prometheus (hoặc tên gì cũng được).
- URL: **http://prometheus:9090** (quan trọng: dùng tên service prometheus vì chúng cùng Docker network).
- Access: chọn Server (mặc định).
- Scrape interval: 15s (khớp với prometheus.yml).

Cuộn xuống dưới và nhấn Save & Test. Nếu hiện "Data source is working" → thành công!

Import dashboard có sẵn. Sau khi thêm Data Source, ta nên import các dashboard đẹp và sẵn dùng, các dashboard khuyến nghị như:

Bảng 7.3: Dashboard Grafana khuyến nghị

Mục tiêu	Dashboard ID	Link import
Node Exporter (host)	1860	Grafana.com
cAdvisor / Docker	14282 hoặc 193	Grafana.com
MySQL	7362	Grafana.com
MySQL Overview	11375	Grafana.com

Cách import:

- Bên trái chọn Dashboards → New → Import.
- Dán ID vào ô (ví dụ: 1860) → Load.
- Chọn Data Source là Prometheus vừa tạo.
- Nhấn Import.

Câu hỏi thảo luận

Câu 1: Tại sao mô hình Pull của Prometheus lại tối ưu hơn mô hình Push?

Câu 2: So sánh sự khác biệt giữa Alerting trong Prometheus (thông qua Alertmanager) và Alerting trực tiếp trên giao diện Grafana. Khi nào nên dùng loại nào?

Câu 3: Tại sao Prometheus lại sử dụng cơ sở dữ liệu chuỗi thời gian (TSDB) thay vì các cơ sở dữ liệu quan hệ truyền thống như MySQL để lưu trữ metrics?

Câu 4: Hãy tìm kiếm trên trang chủ Grafana Dashboard và liệt kê ID của 3 dashboard phổ biến nhất để giám sát Docker Containers.

Bài thực hành

Bài 1. *Cách thiết lập Alert (cảnh báo) trong Grafana?*

Bài 2. *Cấu hình và tích hợp thêm các dịch vụ mới để giám sát với Grafana*